

API Workflow Guide

This document explains how a request moves through the API and how the project pieces connect.

Big Picture

Think of the workflow as a chain. An API endpoint is the combination of an HTTP method and a URL path.

```
Client
|
| HTTP request
v
Uvicorn web server
|
v
FastAPI application
|
v
APIRouter
|
v
Endpoint function in routes.py
|
v
Pydantic validation
|
v
Business logic
|
v
JSON response
```

The Project Layers

Layer	Responsibility
routes.py	Defines HTTP endpoints and handler functions
Pydantic models	Validate request and response data
app/src/main.py	Creates the FastAPI app and includes routers
Uvicorn	Runs the Python web application
Docker	Packages the application
Kubernetes	Runs and exposes application containers
Ollama or Bedrock	Generates the AI answer

1. Client Sends A Request

A client can be a browser, frontend application, curl, Postman, another backend service, or a Kubernetes health probe.

```
GET /health
GET /ready
POST /ask
```

For POST /ask:

```
{
  "question": "What is Kubernetes?"
}
```

2. Uvicorn Receives The Request

Uvicorn is the web server process. It listens for HTTP requests and passes them to FastAPI.

```
\.\.venv\Scripts\python.exe -m uvicorn app.src.main:app --reload
```

app.src.main:app means app.src.main is app/src/main.py and app is the FastAPI variable inside it.

3. FastAPI Maps The Request To A Route

The current route definitions are in `app/src/api/routes.py`.

Request	Handler	Purpose
GET /health	health_check()	Confirms the process is alive
GET /ready	readiness_check()	Confirms the app is ready
POST /ask	ask_question()	Accepts a question and returns an answer

```
@router.get("/health")
def health_check():
    return {"status": "healthy"}

GET /health -> health_check()
```

The route is the public HTTP contract that points to the Python function.

```
GET /health -> health_check()
GET /ready -> readiness_check()
POST /ask -> ask_question()
```

Who Uses Each Endpoint?

Regular users normally do not call `/health` or `/ready` directly. They use the business endpoint:

```
User -> Frontend -> POST /ask -> FastAPI -> AI model
```

Kubernetes, load balancers, and monitoring systems use the health endpoints:

```
Kubernetes -> GET /health
Kubernetes -> GET /ready
```

/health: liveness check

GET `/health` answers: "Is the API process alive?" It should be a small and fast check. If it fails repeatedly, Kubernetes can restart the API container.

```
/health = restart me if the process is broken
```

/ready: readiness check

GET /ready answers: "Can this API safely receive traffic?" If it fails, Kubernetes keeps the pod running but temporarily removes it from Service traffic.

Later, this endpoint can check dependencies such as Ollama, a database, or AWS Bedrock:

```
/health -> 200 OK    (the process is alive)
/ready  -> 503       (a required dependency is unavailable)
```

This prevents users from being routed to a pod that is running but not able to complete requests.

4. Pydantic Validates The Request

```
JSON request
|
v
QuestionRequest
|
v
field_validator("question")
|
v
ask_question(request)
```

The validator removes leading and trailing spaces, rejects an empty question, and rejects text longer than 1,000 characters. Invalid input produces a validation error before the handler runs.

How Does The Question Propagate?

The current implementation receives and validates the question, but it does not send it to an AI service yet:

```
Client
| POST /ask
| {"question": "What is Kubernetes?"}
v
FastAPI route: ask_question(request)
|
| request.question
v
QuestionRequest.question
|
v
temporary_answer <- current stopping point
|
v
QuestionResponse
|
v
Client receives JSON
```

The important line that reads the user's question is `request.question`. The current code does not contain an Ollama URL, Bedrock client, model ID, or AI service call, so the response uses `model: "not-configured"`.

Future propagation after AI integration

```

Client
|
| POST /ask
v
FastAPI route
|
| ai_service.generate_answer(request.question)
v
AI service layer
|
+--> Ollama: HTTP request to http://localhost:11434
|
+--> Bedrock: boto3 API call to AWS
v
Generated answer
|
v
QuestionResponse
|
v
Client

```

The public endpoint remains POST /ask; only the internal provider changes.

Configuration ownership

```

routes.py
  = HTTP endpoint, request validation, and response mapping

config.py or environment variables
  = provider, URL, model name, region, and timeouts

ai_service.py
  = common generate_answer(question) interface

ollama_client.py / bedrock_client.py
  = provider-specific implementation

Kubernetes Service
  = network path to the API Pod; it does not call the AI model itself

```

5. The Handler Performs The Work

FastAPI matches POST /ask to ask_question(), validates the request, runs the handler, and prepares the response.

```
POST /ask
Content-Type: application/json

{
  "question": "What is Kubernetes?"
}
```

6. Handler And Response Relationship

The handler currently returns a temporary answer because an AI model is not connected yet.

```
ask_question()
|
v
Ollama or AWS Bedrock
|
v
Generated answer
|
v
QuestionResponse
```

```
class QuestionResponse(BaseModel):
    question: str
    answer: str
    model: str
```

Example response:

```
{  
  "question": "What is Kubernetes?",  
  "answer": "The API received your question successfully. The AI model is not  
connected yet.",  
  "model": "not-configured"  
}
```

7. Application Wiring

The route file creates an APIRouter, but the FastAPI application must include that router.

The FastAPI application is in `app/src/main.py` and already includes the router.

```
from fastapi import FastAPI  
from .api.routes import router  
  
app = FastAPI(title="Secure AI Platform API")  
app.include_router(router)
```

`include_router(router)` connects the route definitions to the running application. Without it, the functions exist in Python but are not exposed as URLs.

```
routes.py  
-> router = APIRouter()  
-> @router.get(...) and @router.post(...)  
-> app/src/main.py imports router  
-> app.include_router(router)  
-> Uvicorn runs app.src.main:app  
-> clients call the URLs
```

Useful URLs:

```
http://localhost:8000/health  
http://localhost:8000/ready  
http://localhost:8000/docs
```

8. Kubernetes Relationship

Kubernetes does not create the application endpoints. FastAPI creates them. Kubernetes runs the application and routes traffic to it.

```
Browser or frontend
|
v
Kubernetes Service
|
v
API Pod
|
v
Uvicorn
|
v
FastAPI app
|
v
Route handler
```

```
GET /health -> liveness: is the process alive?
GET /ready  -> readiness: should traffic be sent here?
```

Future AI Flow

```
POST /ask
-> FastAPI route
-> AI service client
-> Ollama locally OR AWS Bedrock in AWS
-> generated answer
-> QuestionResponse
```

Ollama is useful for local development. AWS Bedrock is the managed AWS option for production model access.

Automated Testing

Manual browser or PowerShell checks are useful, but repeating them after every code change is inefficient. Automated tests check the API consistently before Docker or Kubernetes deployment.

The tests verify `/health`, `/ready`, valid and invalid `/ask` requests, and the response structure. This is called **shift-left testing**: find problems early, before deployment.

Without tests: Code change -> Docker image -> Kubernetes deployment -> failure discovered late

With tests: Code change -> test detects problem -> pipeline stops

The tests are in `app/tests/test_api.py`. They use FastAPI's `TestClient`, which loads the application directly in memory instead of using a network port.

Test	What it protects
<code>test_health_endpoint</code>	<code>/health</code> returns 200 and healthy status
<code>test_readiness_endpoint</code>	<code>/ready</code> returns 200 and ready status
<code>test_ask_endpoint_with_valid_question</code>	Valid question returns an answer
<code>test_ask_endpoint_removes_extra_spaces</code>	Surrounding spaces are cleaned
Invalid-input tests	Blank, missing, or wrong-type questions return 422

Tests follow three stages:

Arrange -> prepare request data
Act -> call the endpoint
Assert -> check the result

```
request_body = {"question": "What is the policy?"}
response = client.post("/ask", json=request_body)
assert response.status_code == 200
assert response.json()["answer"]
```

Stop Uvicorn with `Ctrl+C` if it is running. From the repository root, with `(.venv)` active, run:

```
python -m pytest .\app\tests -v
```

Expected result:

7 passed

FastAPI code -> pytest -> Docker image -> Kubernetes deployment

Simple Mental Model

Term	Meaning
Route	Public HTTP contract
Handler	Python function that performs the work
Model	Shape and validation of data
Uvicorn	Server process that listens for requests
FastAPI app	Object that owns the registered routes
Kubernetes	Platform that runs and exposes the server

Debug order: Is the server running? Is the URL and HTTP method correct? Is the router included? Does validation accept the body? Does the handler complete? Does Kubernetes route to the correct Pod?